Offline-First Pharmacy Management System

System Design — Nuvotech · Deployment context: DRC · Version 1.0 draft · Oct 6, 2026 · @Demto

1. Overview and constraints

The counter tablet runs a small **sales-only app** that works fully offline. Everything else (stock management, receptions, alerts, dashboard, reports) is a **cloud web app** used from the back-office laptop and tablet. Stock lives in batches (lots), each with its own expiry date and purchase price, which drives FEFO, expiry alerts and stock valuation.

Parameter	Value
Counter	1 tablet with a sales-only app, used by the cashier
Back office	Laptop + tablet using the web app in a browser
Product lookup	Fast search by name or DCI first; barcode scanner optional
Internet	Can drop for hours at the counter; always available in the back office when stock is received
Payment (v1)	Cash in CDF or USD, plus mobile money recorded manually with a reference
Receipts	Printed on request at the counter, never blocks a sale
Users	Cashier (counter app only); pharmacist/admin (back office, all functions)

Core principle: every counter action (search, add to cart, FEFO lot selection, sale) completes against the tablet's local SQLite with zero network calls. The back office is online-only by design, which is acceptable because stock is only received when internet is available.

Out of scope for v1: prescriptions and patient records, insurance billing, controlled-substance registers, multi-branch stock. A simple "prescription required" flag on the medicine is enough to remind the seller.

2. Technology selection

Two clients on one Laravel backend: a native sales app at the counter and a web app for the back office. The counter app reuses the clothing POS shell and sync code.

Layer	Choice	Why
Counter app	React Native (Expo), sales screens only	Native storage survives power cuts on low-end Android; the cashier sees nothing but selling.
Counter database	SQLite (expo-sqlite, WAL) holding a copy of medicines, lots, prices and the day's rate	A sale and its lot deductions commit in one transaction.
Search	SQLite FTS5 on name, DCI and barcode	Instant results while typing, offline.
Receipt printer	Bluetooth ESC/POS, on demand	Decoupled from the sale.
Back-office app	Laravel 11 + Inertia + React, in the browser	Nothing to install on the laptop or tablet; the team's main stack.
Auth	Laravel Sanctum: device token for the counter tablet, user login for the back office	Simple and already familiar.
Database	MySQL on a DigitalOcean droplet	Single source of truth for stock.

3. Data model

Seven core tables. Purchase price and expiry sit on the **batch**; selling price and minimum stock sit on the **medicine**. All IDs are UUIDs generated on the device, so records created offline never collide.

Table	Key fields	Notes
medicines	name, dci, dosage, form, category, barcode (nullable), units_per_pack, sale_price_pack, sale_price_unit, min_stock (units), prescription_required, default_supplier_id	Master data. Edited by the pharmacist only.
suppliers	name (laboratory or wholesaler), phone, address	Also used for supplier returns.
batches	medicine_id, lot_number, expiry_date, purchase_price, supplier_id, qty_received, qty_on_hand, received_at	One row per lot received. qty_on_hand is a cached total of its movements.
stock_movements	batch_id, medicine_id, type, qty (+/−), reason, reference_id, user_id, device_id, created_at	Append-only. This table **is** the movement history and the sync event log.
receptions	supplier_id, invoice_ref, received_at, total_cost, user_id	Header for a delivery; its lines are batches + reception movements.
sales	receipt_no, total (base currency), currency_paid, exchange_rate, amount_paid, payment_method, payment_ref, user_id, device_id, created_at	One per dispensing.
sale_items	sale_id, medicine_id, batch_id, qty (units), sell_mode (pack/unit), unit_price, line_total	One row per lot used — a sale of 10 units can produce two lines if FEFO splits across lots.

Movement types: `reception`, `sale`, `customer_return`, `supplier_return`, `adjustment`, `expiry_writeoff`, `inventory_correction`. Stock is never edited directly — every change is a movement, so history is complete by construction.

4. FEFO and stock movements

The seller never picks a lot — the app does it automatically at checkout, earliest expiry first, and expired lots can never be sold.

FEFO allocation at sale time (one SQLite transaction):

1. Select the medicine's batches where `qty_on_hand > 0` and `expiry_date > today`, ordered by `expiry_date ASC`.
2. Take units from the first batch; if it runs out, continue to the next until the requested quantity is filled.
3. Write one `sale_items` row and one `sale` movement per batch used, then update the cached `qty_on_hand`.
4. If total sellable stock is lower than the quantity asked, block the line and show what is available.

The cart shows the lot and expiry being used, and flags it orange if it expires within the alert window, so the seller can warn the patient.

Other movements (all in the back-office web app):

- **Reception** — enter supplier and invoice ref, then for each line: medicine, lot number, expiry, quantity, purchase price. Creates a batch and a `reception` movement. Optionally update the medicine's sale price.
- **Customer return** — select the original sale; units go back into the same batch if sealed and not expired, otherwise recorded as a write-off.
- **Supplier return** — select batch and quantity; negative movement with reason.
- **Adjustment** — breakage, loss, theft; requires a reason and pharmacist PIN.
- **Expiry write-off** — one tap on the expired-products list moves the whole remaining lot out of stock.
- **Inventory (physical count)** — the system lists every batch; staff enter the counted quantity; differences become `inventory_correction` movements in one go. Best done at closing time when no sales are running.

5. Alerts, dashboard and reports

Alerts, the dashboard and all reports live in the back-office web app, computed from the server database. The counter app only does what selling needs: it never offers expired lots and flags near-expiry lots in the cart.

Alerts (sellable stock excludes expired lots):

Alert	Rule
Out of stock (rupture)	Sellable qty = 0
Low stock	Sellable qty ⩽ min_stock
Expiring soon	A lot with stock expires within 90 days (configurable)
Expired	A lot with stock has expiry_date ⩽ today — blocked from sale, waiting for write-off

Dashboard: number of medicines, stock value at purchase price (sum of qty_on_hand × purchase_price per lot) and at sale price, out-of-stock count, expiring-soon count, today's entries and exits, today's sales total. The counter app shows only the cashier's own sales for the day, for cash-up.

Reports:

Report	Notes
Stock status	Per medicine, with lots and expiries
Expired medicines	With value lost at purchase price
Expiring soon	Sorted by expiry date
Inventory	Printable count sheet and count results
Sales	By day, cashier, payment method and currency
Purchases	By supplier and period
Monthly consumption	Units out per medicine per month, sales and write-offs separated
Most used medicines	Top N by units and by revenue

Counter sales made offline reach these figures when the tablet next syncs; the dashboard shows the counter's last-sync time so a stale figure is never mistaken for a live one. All reports export to PDF and Excel.

6. Sync, trade-offs and risks

The server is the source of truth for stock. The counter tablet keeps a local copy so it can sell offline, sends its sales up, and pulls back the latest stock and prices.

- **Up:** sales and their lot deductions, pushed in batches on reconnect plus a "Sync now" button; the server de-duplicates by UUID.
- **Down:** medicines, prices, lots with current quantities and the exchange rate, pulled after every upload and every few minutes while online, so a new delivery reaches the counter almost immediately.
- **Conflicts:** only possible when the back office changes a lot the counter sold from offline (write-off, supplier return, inventory correction). The server moves the shortfall to the next FEFO lot of the same medicine; if none covers it, it goes to a discrepancy list for the pharmacist.
- **Restore:** a replacement tablet logs in and downloads everything; only unsynced sales on the lost tablet are at risk.

Risk	Decision
Counter offline for hours	Keeps selling from its local copy; the back office sees those sales after sync.
Back-office internet down	Back office waits; acceptable because stock is only received with internet. The counter is unaffected.
Delivery received while the counter is offline	Counter keeps selling existing lots and picks up the new lots on reconnect.
Inventory count while the counter sells	Count after closing, once the counter has synced.
Counter tablet lost or broken	Sync at closing time every day; a spare tablet restores from the server.
Selling an expired lot	Impossible: the counter's FEFO query excludes expired lots.
Wrong tablet date	Tablet compares its clock with the server on each sync and blocks sales if it is clearly wrong.
Power cut mid-sale	SQLite transaction: the sale and all its lot deductions commit together or not at all.

Risk	Decision
Wrong expiry typed at reception	Form rejects past dates and asks to confirm expiries under 6 months.

7. Build order and open questions

Build the back office first: it holds the stock, and the counter app needs that data before it can sell.

1. **Phase 1 — Back-office web app:** medicines, suppliers, opening-stock import, receptions with lots, exchange rate, alerts, dashboard.
2. **Phase 2 — Counter app:** search, cart, FEFO, box/unit and CDF/USD selling, offline storage, sync up and down. Reuse the POS sync module.
3. **Phase 3 — Stock operations:** returns, adjustments, expiry write-off, inventory count, discrepancy list, on-demand receipt printing.
4. **Phase 4 — Reports:** sales, purchases, monthly consumption, most used, PDF/Excel export.

Open questions for the client:

- [x] ~~Selling currency: CDF only, USD only, or both with a daily exchange rate?~~
- [x] ~~Can medicines be sold by unit (tablet/blister) as well as by box? If yes, each medicine needs a box-to-unit conversion.~~
- [ ] How many sales stations, and is there a separate back-office person for receptions?
- [x] ~~Expiry alert window: 90 days, or different per category?~~
- [x] ~~Existing stock list (Excel?) to import on day one, with lot numbers and expiries.~~

Confirmed design changes

Currency — CDF and USD with a daily rate.

- All prices are stored in one base currency, chosen at setup (USD is the natural choice if suppliers invoice in USD).
- The pharmacist enters the day's rate each morning in the back office; the counter picks it up on its next sync. If no rate was entered today, sales use the last one and a warning banner shows.
- The cart shows the total in both currencies; the seller picks the currency paid and change can be given in either.

- Each sale stores the currency paid, the rate used and the amount paid, so reports stay correct when the rate moves.
- New table: `exchange_rates` (date, rate, user_id).

Selling by box or by unit.

- Stock is held in the smallest unit (tablet, capsule, ampoule); each medicine has `units_per_pack`. Items never split (syrups, creams) use 1.
- Two selling prices per medicine: per pack and per unit.
- Receptions are entered in packs and converted to units automatically.
- In the cart the seller toggles Box / Unit; FEFO runs unchanged on units.
- Stock displays as "4 boxes + 12 units"; minimum stock and alerts work in units.

Expiry alert window — 90 days for all medicines, as one setting the pharmacist can change.

Opening stock import from Excel/CSV.

- Template columns: name, DCI, dosage, form, supplier, lot number, expiry, quantity, purchase price, pack price, unit price, units per pack, min stock.
- Imported through the back-office web app in Phase 1: every row is validated and errors are shown before anything is saved.
- Rows become batches with an `opening_stock` movement, so history starts clean.
- Expiry is mandatory (FEFO and alerts depend on it); a missing lot number gets a generated placeholder.

Setup confirmed: one counter tablet with the sales-only app; back office on a laptop and a tablet through the web app.